Ramazan Kozhabek

## 1. Project Proposal

**1.1 Project title**: Development and Research of a Cascaded NLP System for Language Identification, Code-Switching Detection, and Sentiment Analysis of Kazakh-Russian Texts

**1.2 Team members:**Ramazan Kozhabek(Individual Project)

**1.3 Supervisor:** Shynar Akhmetzhanova

**1.4 Problem statement:**

Kazakh and Russian are widely used in Kazakhstan, and bilingual speakers may alternate between these languages within a single sentence or text. This phenomenon, known as code-switching, creates challenges for Natural Language Processing (NLP) systems because the language of a text cannot always be determined at the sentence level. Both languages also use Cyrillic script and share many characters, which can make language identification more difficult

Language identification, code-switching detection, and sentiment analysis are commonly treated as separate NLP tasks. However, processing mixed-language text may require these tasks to be considered together. For example, an incorrect language prediction may affect the detection of language boundaries, while language mixing, context, and linguistic variation may influence sentiment classification.

The main problem addressed by this project is how to design and evaluate a cascaded NLP system capable of processing Kazakh, Russian, and code-switched texts through multiple connected stages. The proposed system will investigate language identification, code-switching detection, and sentiment analysis within a unified processing pipeline.

The project will compare appropriate methods and models for the individual stages and evaluate the overall system using suitable datasets and performance metrics. Particular attention will be given to classification accuracy, the effect of errors between pipeline stages, and the limitations associated with available data and language resources.

### 1.5 Target Users

The proposed system is intended for individuals and organizations that interact with Kazakh, Russian, and mixed-language text.

**Primary target users**

1. **Bilingual and partially bilingual citizens of Kazakhstan.** People who are more fluent in one language than the other may mix Kazakh and Russian words within a sentence when they cannot immediately recall a word in the language they are using. The proposed system aims to process such mixed-language text without requiring users to rewrite their messages in a single language.

2. **Kazakh and Russian language learners.** Students learning either language may use words from their native or stronger language when they forget vocabulary or struggle to express an idea. The system could support language-learning platforms by processing mixed-language input and identifying the languages used within a text.

**Secondary target users**

3. **Businesses and customer support providers.** Companies in Kazakhstan may receive customer messages containing a mixture of Kazakh and Russian. The proposed system could be integrated into business chatbots or customer support platforms to help process mixed-language messages and analyze customer sentiment.
4. **Developers of conversational AI systems.** Developers building chatbots, virtual assistants, and other text-based applications could use the system as an NLP component to identify languages, detect code-switching, and analyze sentiment in user messages.
5. **Educational technology providers.** Developers of language-learning applications and digital educational platforms could use the system to support applications that process mixed Kazakh–Russian text.

The initial research will focus on the technical feasibility and performance of the cascaded NLP pipeline. Its potential applications for these user groups will be considered when defining requirements and evaluating the results.

### 1.6 Project Aim

The aim of this project is to develop and evaluate a cascaded Natural Language Processing (NLP) system capable of identifying languages, detecting code-switching, and analyzing sentiment in Kazakh, Russian, and mixed Kazakh–Russian texts. The project seeks to investigate the effectiveness of combining multiple NLP components into a unified pipeline for processing multilingual text, with potential applications in conversational AI, business chatbots, and language-learning platforms.

### 1.7 Project Objectives

To achieve the aim of the project, the following objectives have been established:

1. **Conduct a literature review** of existing NLP approaches for language identification, code-switching detection, and sentiment analysis, with particular attention to Kazakh–Russian texts.
2. **Collect and prepare datasets** suitable for training and evaluating models for language identification, code-switching detection, and sentiment analysis.

3. **Investigate and compare machine learning and NLP approaches** to identify suitable models for each stage of the proposed system.
4. **Develop individual NLP components** for identifying Kazakh and Russian languages, detecting language switches within text, and classifying sentiment.
5. **Integrate the developed components into a cascaded NLP pipeline** that processes input text and produces language identification, code-switching, and sentiment analysis results.
6. **Evaluate the performance of individual components and the complete pipeline** using appropriate metrics, such as precision, recall, F1-score, and accuracy where applicable.
7. **Analyze the limitations of the proposed system**, including the effects of errors between pipeline stages, data availability, and linguistic variation in mixed Kazakh–Russian texts.
8. **Investigate potential applications** of the system in conversational AI, business chatbots, and language-learning platforms.

### 1.8 Expected Outcomes

The expected outcomes of this project are:

1. **Prepared datasets** for language identification, code-switching detection, and sentiment analysis of Kazakh, Russian, and mixed Kazakh–Russian texts.
2. **Developed NLP models** capable of identifying the language of text, detecting language switches, and classifying sentiment.
3. **An integrated cascaded NLP system** that combines the developed components into a unified pipeline for processing Kazakh–Russian text.
4. **A performance evaluation report** containing quantitative results for individual models and the complete pipeline, using appropriate evaluation metrics such as precision, recall, F1-score, and accuracy where applicable.
5. **A comparative analysis of NLP approaches** that examines model performance, limitations, and suitability for processing mixed-language text.
6. **Documented technical architecture and implementation**, including the system design, selected technologies, testing procedures, and instructions for running the system.
7. **An analysis of potential practical applications** in conversational AI, business chatbots, and language-learning platforms, including the system's current capabilities and limitations.

### 1.9 Project Scope

The scope of this project covers the design, development, integration, and evaluation of a cascaded Natural Language Processing (NLP) system for processing Kazakh, Russian, and mixed Kazakh–Russian texts. The project consists of three main components: the NLP model pipeline, a REST API, and a native web interface.

**In Scope**

1. NLP Model Development. Development and evaluation of NLP components for language identification, code-switching detection, and sentiment analysis. The project will investigate suitable models and methods for each task, prepare relevant datasets, and evaluate model performance using appropriate metrics.

2. Cascaded NLP Pipeline. Integration of the developed NLP components into a unified processing pipeline in which the output of one stage can be used by subsequent stages. The pipeline will process input text and generate language identification, code-switching detection, and sentiment analysis results.

3. REST API Development. Implementation of a REST API using FastAPI to provide programmatic access to the NLP pipeline. The API will accept text input, process it through the integrated system, and return structured results, including language identification, code-switching information, sentiment predictions, and relevant confidence scores where supported. API documentation will be provided through the framework's built-in documentation interface.

4. Native Web Interface Development. Development of a web interface that allows users to enter Kazakh, Russian, or mixed-language text and view the results produced by the NLP system. The interface will present language identification, code-switching detection, and sentiment analysis results in an understandable format. It will communicate with the backend through the REST API and provide a practical way to test the system without requiring users to interact with the API directly.

5. System Integration and Testing. Integration of the NLP pipeline, REST API, and web interface into a functioning system. Testing will cover the correctness of API responses, communication between system components, interface functionality, and the processing of different types of input text.

6. Performance Evaluation. Evaluation of individual NLP components and the complete pipeline using appropriate metrics, including accuracy, precision, recall, and F1-score where applicable. The evaluation will also examine error propagation between pipeline stages and the system's limitations when processing code-switched text.

7. Technical Documentation. Preparation of documentation describing the system architecture, model development, selected technologies, API endpoints, installation and execution procedures, and testing and evaluation results.

**Out of Scope**

The following features and activities are outside the scope of the current project:

1. Development of a mobile application or native desktop application.

2. Integration with third-party messaging platforms, commercial chatbots, or external educational platforms.
3. Development of a complete language-learning platform, business customer support system, or conversational AI assistant.
4. Implementation of user accounts, authentication, payment processing, or personalized user profiles unless required for basic system functionality.
5. Development of real-time voice recognition, speech-to-text processing, or audio-based sentiment analysis.
6. Training a large language model from scratch or developing a general-purpose multilingual AI model.
7. Production-scale deployment, high-availability infrastructure, and large-scale distributed processing.

**Scope Limitations**

The system will focus specifically on Kazakh, Russian, and mixed Kazakh–Russian text. Its performance will depend on the availability, quality, and representativeness of the datasets used for training and evaluation. The project will prioritize the technical feasibility and measurable performance of the NLP pipeline rather than commercial deployment or integration into existing third-party platforms. The web interface and REST API will serve as two access methods to the same underlying NLP system rather than independent implementations of the models.

## 2. Evidence of the relevance

### 2.1. Research Methodology

To identify potential requirements for the proposed NLP system, four semi-structured interviews were conducted with individuals representing different language proficiency backgrounds and potential application contexts. The interviews were conducted in English and focused on experiences with Kazakh–Russian communication, difficulties in understanding mixed-language text, and the potential usefulness of automated language identification, code-switching detection, and sentiment analysis.

The interviews were documented using audio recordings and written transcripts. The responses were reviewed to identify relevant problems, potential use cases, and requirements for the proposed system.

### 2.2. Interview Participants

Four participants were selected to provide different perspectives on the proposed system.

- **Participant 1:** An individual with limited Kazakh proficiency who primarily communicates in Russian.
- **Participant 2:** An individual with limited Kazakh proficiency who encounters mixed-language content in online communication.
- **Participant 3:** An individual with limited Russian proficiency who primarily communicates in Kazakh.
- **Participant 4:** A representative interested in the potential integration of the proposed NLP model into an educational platform operating in Kazakhstan.

The participants were anonymized using identification codes to maintain confidentiality. The interviews were conducted in English to ensure consistent documentation and analysis.

### 2.3. Interview Questions

The interviews were guided by three questions:

1. How does your level of proficiency in Kazakh and Russian affect your ability to understand or write texts that contain both languages?
2. Have you encountered situations where identifying the language or understanding the sentiment and tone of a mixed Kazakh–Russian text was difficult? Please describe an example.
3. Would an NLP system that identifies languages, detects code-switching, and analyzes sentiment and tone be useful to you? How would you use it?

These questions were designed to explore language-related difficulties, identify practical use cases, and assess the potential relevance of the proposed system.

### 2.4. Interview Findings

The interviews explored several potential applications of the proposed system. Participants with limited proficiency in one of the two languages described difficulties they encounter when reading mixed-language text, particularly when unfamiliar expressions or changes between languages affect their understanding of a message.

The discussions also highlighted the potential value of sentiment for interpreting online communication. Language identification and code-switching detection could help distinguish the languages used within a text, while sentiment predictions could provide additional information about its emotional meaning.

The fourth participant identified a potential application in educational technology. Integrating the NLP model into an educational platform could allow the platform to analyze student feedback and comments written in Kazakh, Russian, or a mixture of the two languages. Such functionality could support the analysis of feedback and help identify general sentiment trends.

### 2.5. Implications for System Requirements

The interview topics informed the identification of several functional requirements for the proposed system:

- **Language identification:** Determine whether the submitted text is in Kazakh, Russian, or a supported combination of the two languages.
- **Code-switching detection:** Identify whether a text contains both Kazakh and Russian.
- **Sentiment analysis:** Classify the overall sentiment of the submitted text using the sentiment categories supported by the trained model.
- **API integration:** Provide a REST API that allows external applications, including educational platforms, to submit text and receive structured analysis results.

These requirements will guide the subsequent design, implementation, and evaluation of the NLP system.

### 2.6. Interview Documentation

The original interview recordings are stored in Google Drive, and written transcripts are maintained as supporting research documentation. Each interview is associated with a participant identification code to link the transcript and recording without including unnecessary personal information.

The full interview transcripts are provided in **Appendix A**, while links to the corresponding audio recordings are provided in **Appendix B**. These materials serve as supporting evidence for the interview findings presented in this section and are available for review by the project supervisor.

## 3. Literature Review

### 3.1. Overview of Multilingual Natural Language Processing

Natural Language Processing (NLP) enables computer systems to process, classify and interpret human language. Important NLP tasks include language identification, sentiment analysis, text classification and the analysis of multilingual communication. Although substantial progress has been made in these areas, processing text that contains multiple languages remains challenging because languages differ in their vocabulary, morphology, syntax and writing conventions. These difficulties are particularly relevant to multilingual communities in which speakers frequently alternate between languages within the same sentence or conversation.

Traditional NLP approaches commonly rely on manually designed features, statistical methods and task-specific classifiers. More recent approaches use neural networks and pretrained Transformer models to learn contextual representations of words and sentences. Such models can be adapted to different classification tasks through fine-tuning, reducing the need to train a complete model from scratch. However, their effectiveness depends on the languages represented during pretraining, the quality of available training data and the similarity between the training and evaluation datasets.

Conneau et al. [1] introduced XLM-RoBERTa (XLM-R), a multilingual Transformer model pretrained on text from 100 languages. Their experiments demonstrated the benefits of multilingual pretraining for cross-lingual transfer and several NLP tasks. Nevertheless, strong performance on multilingual benchmarks does not necessarily guarantee reliable processing of informal, mixed-language text. Khanuja et al. [2] demonstrated the importance of evaluating multilingual models specifically on code-switched data rather than relying exclusively on monolingual benchmarks.

These findings provide the theoretical foundation for the present diploma project, which investigates a cascaded NLP system for language identification, code-switching detection and sentiment analysis of Kazakh–Russian text. The following subsections examine existing approaches to these tasks, their reported results and the limitations that influence the proposed system's design.

### 3.2. Multilingual Transformer Models and Cross-Lingual Transfer

Multilingual pretrained language models are particularly relevant to NLP applications involving languages with different levels of digital-resource availability. Instead of learning language representations exclusively from a small task-specific dataset, a model can first learn general linguistic patterns from large multilingual corpora and subsequently be fine-tuned on a smaller labelled dataset.

Conneau et al. [1] presented *Unsupervised Cross-lingual Representation Learning at Scale*, introducing XLM-R. The model uses a Transformer encoder and masked language modelling to learn contextual representations from multilingual text. The authors reported improvements over multilingual BERT on several cross-lingual benchmarks, demonstrating that large-scale multilingual pretraining can support knowledge transfer between languages. This approach is relevant to Kazakh–Russian NLP because it provides a way to reuse pretrained representations when task-specific annotated data is limited.

However, multilingual pretraining has limitations. Languages may be represented unequally in the training corpus, and a model's performance can vary across languages and tasks. Furthermore, multilingual models are not necessarily trained to recognise language changes within individual sentences. Therefore, selecting a multilingual encoder should be followed by task-specific fine-tuning and evaluation on data representative of the intended application.

Khanuja et al. [2] introduced GLUECoS, an evaluation benchmark for code-switched NLP covering English–Hindi and English–Spanish. The benchmark includes language identification, sentiment analysis and other language-processing tasks. Their experiments showed that multilingual models fine-tuned on code-switched data generally performed better than approaches relying only on cross-lingual representations. This finding suggests that exposure to mixed-language examples during training is important for reliable code-switched text processing.

A related study by Ranjan et al. [3], *Progressive Sentiment Analysis for Code-Switched Text Data*, investigated the imbalance between resource-rich and resource-poor languages in mixed-language datasets. The proposed training strategy progressively

introduces examples according to the proportion of words belonging to the resource-rich language. The experiments indicated that this approach can improve performance on samples dominated by the lower-resource language. This finding is relevant when considering the balance of Kazakh and Russian examples in the training dataset.

Although these studies support the use of multilingual pretrained models, their findings cannot be transferred directly to Kazakh–Russian text because the evaluated language pairs differ. Consequently, the proposed system must be evaluated on its own dataset rather than assuming that results obtained on other language pairs will be reproduced.

### 3.3. Language Identification in Multilingual Text

Language identification is the task of determining which language or languages are present in a text. It is commonly used as an initial stage in NLP pipelines because subsequent processing may depend on the language of the input. For example, a system that processes Kazakh–Russian text may need to distinguish Kazakh-only sentences, Russian-only sentences and sentences containing both languages.

Agarwal et al. [4] proposed LIMIT, a hierarchical approach to language identification across more than 350 languages. They introduced the MCS-350 multilingual corpus and investigated how systematic misclassification patterns could be used to resolve language-identification errors. Their work demonstrates that language identification remains challenging, particularly for languages with limited training resources. It also shows that error analysis and the correction of systematic mispredictions can improve language-identification systems without requiring a completely new large-scale model.

Burchell et al. [5], in *Code-Switched Language Identification is Harder Than You Think*, examined language identification in code-switched text. Rather than treating the problem solely as conventional single-language classification, they investigated approaches that account for the presence of multiple languages and considered sentence-level multilabel prediction. Their results indicated that existing approaches still have limitations when identifying languages in mixed-language text. This distinction is important because identifying the dominant language of a sentence is not equivalent to detecting every language represented within it.

Burchell et al. [6] also presented *An Open Dataset and Model for Language Identification*, introducing a manually audited dataset and a language-identification model covering 201 languages. The authors reported a macro-averaged F1-score of 0.93 and a false-positive rate of 0.033% for their model. These results provide a useful reference for language-identification methodology and evaluation. Nevertheless, the reported results do not establish equivalent performance on Kazakh–Russian code-switching, because general language-identification benchmarks and mixed-language inputs represent different evaluation conditions.

The literature therefore suggests that language identification should be evaluated using more than overall accuracy. Class-level precision, recall and F1-score can reveal whether a model systematically misclassifies one language. For mixed-language input, an additional evaluation should establish whether the system can recognise the presence of both languages rather than assigning the entire sentence to a single class.

In the proposed project, language identification forms the initial stage of the cascaded system. Its output provides information for subsequent processing. However, because language-identification errors can affect later stages, the system should also be tested on short, informal and mixed-language examples.

### 3.4. Code-Switching Detection and Its Challenges

Code-switching refers to the use of two or more languages within a conversation, sentence or utterance. In written communication, this may involve alternating words, phrases or clauses from different languages. Code-switching detection aims to determine whether such mixing occurs, while word-level language identification assigns language labels to individual tokens.

These tasks are related but distinct. A sentence-level classifier may recognise that a sentence contains both Kazakh and Russian without identifying which words belong to each language. Conversely, a word-level language-identification system may assign language labels to individual tokens, allowing the presence and distribution of language changes to be determined.

Khanuja et al. [2] demonstrated that code-switched NLP requires dedicated evaluation across multiple tasks, including language identification and sentiment analysis. Their GLUECoS benchmark provides a useful example of how several code-switched NLP tasks can be evaluated systematically. However, its English–Hindi and English–Spanish datasets do not directly represent the linguistic characteristics of Kazakh–Russian communication.

Burchell et al. [5] further demonstrated that code-switched language identification is more difficult than conventional language identification. Mixed-language inputs introduce ambiguity because a sentence may contain tokens from several languages, and some words may be shared across languages or difficult to classify without context. These issues make it important to define the level at which code-switching is detected and to select evaluation metrics that reflect the intended task.

Another important challenge is data availability. Models trained predominantly on monolingual text may not encounter enough examples of language alternation to learn reliable representations of code-switched sentences. Ranjan et al. [3] showed that the distribution of languages within training samples can influence sentiment-classification performance. Although their primary task was sentiment analysis, the findings also highlight the importance of considering language proportions when preparing mixed-language datasets.

For the proposed Kazakh–Russian NLP system, code-switching detection should therefore be defined explicitly. At the sentence level, the system can classify text as monolingual or mixed-language. If the implementation includes word-level identification, each token can additionally be assigned a language label. The selected output format should correspond to the available annotations and the intended system requirements.

### 3.5. Sentiment Analysis for Kazakh and Multilingual Text

Sentiment analysis identifies the evaluative polarity expressed in text, commonly by assigning positive, negative or neutral labels. It is used in applications such as review analysis, opinion mining and the processing of user-generated content. However, sentiment classification can become more difficult when the input contains multiple languages, informal spelling, slang, negation or context-dependent expressions.

Yeshpanov and Varol [7] introduced KazSAnDRA, a Kazakh sentiment-analysis dataset containing 180,064 reviews. The authors investigated sentiment classification using several machine-learning approaches. The dataset provides an important resource for Kazakh NLP because it supports the development and evaluation of sentiment classifiers in a language that is less extensively represented in NLP research than English.

Nevertheless, a dataset of Kazakh reviews does not automatically represent mixed Kazakh–Russian social-media text. Differences in writing style, domain, language distribution and annotation conventions may affect model performance. Therefore, KazSAnDRA is a useful reference for understanding Kazakh sentiment analysis, but it should not be treated as direct evidence that a model trained on it will accurately classify code-switched sentences.

Sharma et al. [8] proposed *Late Fusion of Transformers for Sentiment Analysis of Code-Switched Data*. Their method combines the outputs of two Transformer models and uses a neural network to produce the final sentiment prediction. Experiments on English–Hindi and English–Spanish datasets demonstrated the potential of combining model outputs for code-switched sentiment analysis. This work provides a relevant architectural comparison for the present project because it investigates how multiple components can contribute to one final prediction.

Etori and Gini [9] introduced RideKE, a dataset and study focused on sentiment and emotion detection in Kenyan code-switched social-media text. Their work illustrates the importance of user-generated content and low-resource multilingual settings. It is relevant to the present project because informal social-media text may contain linguistic patterns that are absent from formal review datasets. However, the language pair and annotation scheme differ from those required for Kazakh–Russian sentiment analysis.

These studies suggest that sentiment analysis should be evaluated using data that reflects the intended language combination and domain. Accuracy alone may conceal poor performance on a minority class; macro-averaged F1-score and class-level metrics provide additional information about model behaviour. The proposed project will therefore need to evaluate sentiment classification separately on monolingual Kazakh, monolingual Russian and mixed-language inputs wherever sufficient labelled examples are available.

### 3.6. Training Strategies for Limited Code-Switched Data

The availability of labelled data is an important factor in developing NLP systems for lower-resource language combinations. Manual annotation requires time and linguistic expertise, while automatically generated examples may introduce errors or fail to

reproduce natural language use. Several studies have investigated ways to address this limitation.

Gupta et al. [10], in *Unsupervised Self-Training for Sentiment Analysis of Code-Switched Data*, proposed a framework that uses pretrained BERT models and pseudo-labels to learn from code-switched data with limited manual annotation. Their experiments showed that unsupervised self-training could approach the performance of supervised models on the evaluated datasets. The work provides an example of how existing pretrained models may be adapted when labelled mixed-language data is scarce.

Samanta et al. [11] investigated *Improved Sentiment Detection via Label Transfer from Monolingual to Synthetic Code-Switched Text*. Their approach creates synthetic code-switched training examples from labelled monolingual text and uses the transferred labels to increase the amount of training data. The authors reported improvements for several language pairs, including English–Hindi, English–Spanish and English–Bengali. This demonstrates the potential of data augmentation when manually annotated code-switched examples are limited.

However, synthetic examples and pseudo-labels may contain incorrect language combinations or inaccurate sentiment labels. Their effectiveness also depends on how closely the generated data resembles real-world text. These approaches should therefore be considered alternatives for future development rather than assumed solutions to the data limitations of Kazakh–Russian NLP.

For the present project, the initial priority is to establish a reliable dataset, define consistent labels and evaluate the model on held-out examples. If the available data proves insufficient, synthetic augmentation or self-training could be investigated as extensions. Such methods would require separate experiments to determine whether they improve performance on authentic Kazakh–Russian text.

### 3.7. Limitations of Current Multilingual Models on Code-Switched Text

The ability to process multiple languages does not necessarily imply the ability to process language switching reliably. This distinction is important when selecting a model for an application that combines language identification, code-switching detection and sentiment analysis.

Khanuja et al. [2] found that multilingual models fine-tuned on code-switched examples generally performed better on the GLUECoS tasks than models relying only on cross-lingual representations. Burchell et al. [5] likewise identified unresolved challenges in code-switched language identification. Together, these findings indicate that dedicated training and evaluation are necessary when applying multilingual models to mixed-language text.

The multilingual Transformer approach proposed by Conneau et al. [1] offers a suitable foundation for the present project because its pretrained encoder can be adapted to different classification tasks. However, its suitability for Kazakh–Russian code-switching must be established empirically. Potential limitations include the availability of

representative labelled data, differences between training and deployment text, confusion between language labels and error propagation between pipeline stages.

A cascaded architecture introduces a further consideration. In a cascade, the output of an earlier stage influences the input or processing decision of a later stage. If language identification incorrectly labels a mixed-language sentence, subsequent code-switching detection or sentiment analysis may be affected. The complete system should therefore be evaluated in addition to its individual components.

A meaningful evaluation should include accuracy and macro-F1 for classification tasks, confusion matrices for language identification and sentiment classification, and separate results for monolingual and code-switched inputs. If the system provides confidence scores, their interpretation should also be assessed rather than treating high confidence as proof of correctness.

### 3.8. Research Gap and Justification for the Proposed System

The reviewed literature establishes several important foundations for the proposed diploma project. First, multilingual pretrained Transformers can support cross-lingual transfer and task-specific classification [1]. Second, benchmarks such as GLUECoS demonstrate the importance of evaluating code-switched text directly [2]. Third, research on language identification shows that recognising multiple languages within the same input remains a distinct challenge [4–6]. Finally, KazSAnDRA provides a valuable foundation for Kazakh sentiment analysis, while studies of code-switched sentiment classification demonstrate the potential of Transformer-based and data-efficient approaches [7–11].

Nevertheless, the reviewed studies differ in their target languages, datasets, tasks and system architectures. Results obtained for English–Hindi, English–Spanish or Kenyan code-switched text cannot establish the performance of a system for Kazakh–Russian input. Likewise, success on Kazakh monolingual sentiment analysis does not by itself demonstrate reliable sentiment classification on mixed-language sentences.

The research problem addressed by this project is therefore the development and empirical evaluation of a cascaded NLP system that combines language identification, code-switching detection and sentiment analysis for Kazakh–Russian text. The project will investigate whether these connected stages can provide useful predictions within one system and identify the limitations that arise from dataset size, language imbalance and errors between stages.

The proposed system will use a multilingual Transformer encoder as the foundation for task-specific prediction. Its effectiveness will be assessed using appropriate classification metrics and examples representing Kazakh, Russian and mixed-language input. The results will be used to examine the strengths and limitations of the selected architecture rather than assuming that performance reported in other language settings will transfer directly.

In summary, the literature supports the feasibility of multilingual text classification and provides methods for addressing code-switching and limited annotated data. At the

same time, it highlights the need for language-pair-specific evaluation and careful analysis of pipeline errors. These observations motivate the development of the proposed cascaded NLP system and define the main issues that its experimental evaluation must address.

## 4.Existing System Comparison

### 4.1. Overview of Existing Systems

Natural language processing (NLP) provides various tools and models for analyzing multilingual text. Existing solutions support tasks such as language identification, machine translation, and sentiment analysis. However, these capabilities are often provided separately, and support for individual languages does not necessarily guarantee reliable processing of sentences containing multiple languages.

This issue is relevant to Kazakh-Russian communication in Kazakhstan, where speakers may combine Kazakh and Russian words within a single sentence. A system designed for this context should be able to identify the languages present in a text, detect language changes within sentences, and determine the sentiment expressed by the text.

Four existing solutions were selected for comparison: Google Translate, Google Cloud Natural Language API, fastText Language Identification, and XLM-RoBERTa. These solutions represent general-purpose language services, cloud-based NLP functionality, lightweight language identification, and multilingual transformer models, respectively.

Google Translate provides machine translation and input-language detection. Google Cloud Natural Language API provides sentiment analysis and other text-analysis functions, although the languages supported depend on the specific feature. The fastText language identification model supports 176 languages, including Kazakh and Russian, and provides a lightweight baseline for language identification [1, 2]. XLM-RoBERTa is a multilingual transformer architecture that can be adapted to downstream NLP tasks through fine-tuning [3].

The selected systems were chosen because their capabilities overlap with at least one component of the proposed solution. Comparing them makes it possible to distinguish between functionality already available in existing tools and functionality that requires further investigation or development.

### 4.2. Comparison Criteria

**The systems are compared using eight criteria:**

1. **Kazakh language support:** Whether the system supports Kazakh for the relevant task.
2. **Russian language support:** Whether the system supports Russian for the relevant task.
3. **Language identification:** Whether the system can identify the language of an input text.
4. **Code-switching detection:** Whether the system can identify changes between Kazakh and Russian within a sentence, including token-level language classification.
5. **Sentiment analysis:** Whether the system can determine whether a text expresses positive, negative, or neutral sentiment**.**
6. **Mixed-language processing:** Whether performance on Kazakh-Russian code-switched text has been documented or experimentally evaluated.
7. **Customization:** Whether the system supports model training, fine-tuning, or adaptation to a specific dataset.
8. **Deployment accessibility:** Whether the system can be executed locally or requires access to an external service.

Official documentation and scientific publications provide evidence for the documented capabilities of the selected systems. Capabilities that are not established by the available documentation are treated as requiring further verification or experimental evaluation.

### 4.3. Comparative Analysis

| Criterion | Google Translate | Google Cloud Natural Language API | fastText Language Identification | XLM-RoBERTa |
|---|---|---|---|---|
| **Kazakh support** | Translation and language detection | Kazakh is not listed among supported sentiment-analysis languages | Supported | Multilingual pretrained model |
| **Russian support** | Translation and language detection | Supported by some features, but not listed for sentiment analysis | Supported | Multilingual pretrained model |
| **Language identification** | Available | Automatic language detection is available | Core functionality | Requires a classification head or another identification method |

| | | | | |
|---|---|---|---|---|
| **Code-switching detection** | No dedicated token-level detection established | No dedicated Kazakh-Russian token-level detection established | Standard model predicts the language of input text; token-level detection requires an additional approach | Can be fine-tuned for token-level classification |
| **Sentiment analysis** | Not its primary function | Available for supported languages | Requires a separate classifier | Requires task-specific fine-tuning |
| **Mixed-language performance** | Requires evaluation for the target language pair | Requires evaluation for the target language pair | Requires evaluation on code-switched text | Requires evaluation on code-switched text |
| **Customization** | Limited control over the underlying translation model | Depends on the available service and feature | Lightweight and adaptable within its supported framework | Supports fine-tuning |
| **Deployment accessibility** | Primarily an online service | Hosted cloud API | Can run locally | Can run locally with suitable computational resources |

*Table 1. Preliminary comparison of existing systems based on their documented capabilities and intended functionality.*

The comparison shows that the selected systems address different aspects of multilingual text processing. Google Translate provides general-purpose translation and language detection, but these capabilities do not establish that it can perform token-level code-switching detection or sentiment classification. Google Cloud Natural Language API offers sentiment analysis, but its official language-support documentation lists different languages for different features. Kazakh is not listed among its supported sentiment-analysis languages [2].

The fastText language identification model provides a practical baseline because it supports both Kazakh and Russian and can be executed locally [1]. Nevertheless, predicting a language for an entire input is different from assigning language labels to individual tokens in a mixed-language sentence. Its performance on the target task must therefore be measured experimentally.

XLM-RoBERTa provides a flexible multilingual representation that can be fine-tuned for classification tasks. However, its pretrained multilingual capabilities do not automatically guarantee accurate Kazakh-Russian code-switching detection or sentiment analysis. Task-specific datasets, fine-tuning, and evaluation are necessary to determine its suitability for the proposed application [3].

Overall, the comparison suggests that the selected systems provide useful individual capabilities, but their documented functionality does not establish that they offer the exact combination of language identification, token-level code-switching detection, and sentiment analysis required by the proposed research. This observation motivates

further investigation rather than proving that no existing solution can perform these tasks.

### 4.4. Research Gap and Justification of the Proposed Solution

The main research problem is the processing of Kazakh-Russian text in which both languages may occur within a single sentence. Although multilingual NLP models and language identification tools already exist, their suitability for this particular combination of languages and tasks must be established through appropriate evidence and evaluation.

The proposed diploma project aims to develop and evaluate a cascaded NLP system for language identification, code-switching detection, and sentiment analysis of Kazakh-Russian text. The system will consist of interconnected processing stages, with the output of one stage potentially providing useful information to subsequent stages.

The first stage will identify the language or languages present in the input. The second stage will detect language changes within the text, potentially assigning language labels to individual tokens. The final stage will classify the sentiment expressed by the input text. The exact implementation of each stage will be determined through model selection and experimental evaluation.

A multilingual transformer model such as XLM-RoBERTa will be investigated as a candidate for the relevant classification tasks. Lightweight approaches, including fastText and TF-IDF with Logistic Regression where appropriate, will provide baseline comparisons. The use of separate models or a shared architecture will be considered according to the requirements and results of each task.

The proposed system will be evaluated using labeled Kazakh, Russian, and Kazakh-Russian code-switched text. Language identification will be evaluated using accuracy and macro-averaged F1-score. Code-switching detection will be evaluated using token-level precision, recall, and F1-score, while sentiment analysis will be evaluated using macro-averaged F1-score and a confusion matrix.

The research will investigate whether integrating the three tasks into a cascaded pipeline provides practical benefits compared with appropriate baseline approaches. It will also examine the effects of code-switching, language imbalance, and dataset size on model performance.

The expected contribution is an experimentally evaluated NLP pipeline tailored to Kazakh-Russian text, together with an analysis of its capabilities and limitations. The proposed solution will not be assumed to outperform existing systems in advance; its value will be established through the results of the experiments.

### 4.5 References

**[1]** Facebook Research, “Language Identification,” *fastText Documentation*. Available: **https://fasttext.cc/docs/en/language-identification.html**

**[2]** Google Cloud, “Language Support,” *Cloud Natural Language API Documentation*. Available**:** **https://docs.cloud.google.com/natural-language/docs/languages**

**[3]** A. Conneau et al., “Unsupervised Cross-lingual Representation Learning at Scale,” *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 8440–8451. Available: https://aclanthology.org/2020.acl-main.747/

## 5. Technology Comparison

### 5.1. NLP Model Architecture

The selection of an appropriate NLP model architecture is an important technical decision because the proposed system must process Kazakh, Russian, and code-switched text. Three approaches are considered: TF-IDF with Logistic Regression, Bidirectional Long Short-Term Memory (BiLSTM), and XLM-RoBERTa.

TF-IDF with Logistic Regression provides a lightweight baseline for text classification. It is computationally efficient and relatively easy to interpret, but it does not directly capture contextual relationships between words. BiLSTM models process sequences of words and can capture contextual information, although they require task-specific training and may be less effective when labeled training data is limited. XLM-RoBERTa is a multilingual transformer model that provides contextual representations and can be fine-tuned for classification tasks.

XLM-RoBERTa is proposed as the primary candidate because of its multilingual capabilities and suitability for adapting to different NLP tasks. TF-IDF with Logistic Regression will be considered as a baseline to evaluate whether the transformer provides meaningful performance improvements. The final model selection will depend on experimental results, dataset characteristics, and computational requirements.

### 5.2. System Architecture

Three architectural approaches are considered for integrating language identification, code-switching detection, and sentiment analysis: independent models, a cascaded pipeline, and a shared multitask model.

Independent models perform each task separately, allowing individual components to be trained and evaluated independently. However, this approach does not explicitly transfer information between tasks. A cascaded pipeline passes the output of one processing stage to subsequent stages, allowing earlier predictions to inform later analysis. Its main limitation is the possibility of error propagation. A shared multitask model uses a common representation for multiple tasks, potentially improving efficiency but increasing the complexity of training and evaluation.

A cascaded pipeline is proposed because it directly corresponds to the project's objective of integrating three related NLP tasks. In the proposed architecture, language identification will provide information about the languages present in the input, code-switching detection will identify language changes within the text, and sentiment analysis will classify the expressed sentiment. The effectiveness of this architecture will

be evaluated using task-specific metrics and comparisons with suitable baseline approaches.

### 5.3. Dataset Storage and Management

Three data-storage approaches are considered: CSV files, SQLite, and PostgreSQL.

CSV files provide a simple format for storing sentences and their corresponding labels. They are compatible with common Python data-processing libraries and are suitable for preparing datasets for model training and evaluation. However, they offer limited support for complex queries and concurrent access. SQLite provides structured data storage in a single database file without requiring a separate database server. PostgreSQL supports more advanced querying and concurrent access but requires additional configuration and maintenance.

CSV files are proposed for the initial research and experimentation stages because the project primarily involves collecting, labeling, preprocessing, and evaluating text samples. Separate datasets can be maintained for language identification, code-switching detection, and sentiment analysis. If the final application requires more complex data management, SQLite or PostgreSQL may be considered according to the system requirements.

### 5.4. Model Serving and Component Interaction

The system must provide a method for connecting the NLP components and, if required, making their functionality available to a user interface or external application. Three approaches are considered: direct Python function calls, Flask, and FastAPI.

Direct Python function calls provide the simplest method of integrating the components within a single application. They introduce minimal overhead and are suitable for local experimentation, but they do not independently provide an HTTP interface. Flask is a lightweight web framework that can expose the system through REST endpoints. FastAPI also supports REST APIs and provides request validation, typed interfaces, and automatic API documentation.

FastAPI is proposed if the final system requires a web interface or integration with other applications. It can expose the cascaded pipeline through an API while keeping the individual NLP components modular. If the project remains a standalone research prototype, direct Python function calls may be sufficient, avoiding unnecessary implementation complexity.

### 5.5. Technology Selection Justification

The proposed technology choices are based on the project's functional requirements, computational resources, and implementation scope. XLM-RoBERTa will be investigated as the primary multilingual model, while TF-IDF with Logistic Regression will provide a lightweight baseline. A cascaded architecture will be used to integrate the three NLP tasks and investigate whether information passed between stages improves the analysis of Kazakh-Russian text.

CSV files will be used for initial dataset management because they provide a straightforward and reproducible approach to storing labeled text samples. FastAPI will be considered for exposing the system through a REST API if the final application requires external access or a user interface.

These choices balance multilingual processing capabilities, research flexibility, and implementation complexity. Their suitability will be assessed through model evaluation and testing against the project requirements. The final implementation may be adjusted if experimental results or practical constraints indicate that an alternative approach is more appropriate.

## 6. Requirements Specification

### 6.1. Functional Requirements

The proposed system will process Kazakh, Russian, and Kazakh-Russian code-switched text using a cascaded NLP architecture based on pretrained XLM-RoBERTa models. The functional requirements define the operations that the system must perform.

| ID | Functional requirement | Priority | Acceptance criteria | Verification method |
|---|---|---|---|---|
| **FR-01** | The system shall accept text containing Kazakh, Russian, or a combination of both languages. | Must | The system accepts valid text input and processes it without errors. | Functional testing |
| **FR-02** | The system shall identify the language of the input text. | Must | The system returns a language label or labels supported by the implemented language-identification component. | Testing against labeled examples |
| **FR-03** | The system shall detect Kazakh-Russian code-switching within a sentence. | Must | The system identifies mixed-language text and produces token-level language labels for supported input. | Comparison with manually labeled test data |
| **FR-04** | The system shall classify the sentiment of the input text. | Must | The system returns one of the implemented sentiment classes: positive, negative, or neutral. | Testing against sentiment-labeled examples |
| **FR-05** | The system shall integrate language identification, code-switching detection, and sentiment analysis into a cascaded processing pipeline. | Must | The three components execute in the defined order and return their respective outputs. | Integration testing |
| **FR-06** | The system shall return structured results containing the input text and the predictions produced by the implemented components. | Must | The output contains the required predictions in a documented format. | Output validation |

| | | | | |
|---|---|---|---|---|
| **FR-07** | The system shall handle empty or invalid input appropriately. | Must | Empty input is rejected or handled gracefully, and an appropriate error message is returned. | Negative testing |
| **FR-08** | The system may provide a user interface for submitting text and viewing analysis results. | Should | If implemented, users can submit text and view the system's predictions. | Interface testing |

### 6.2. Non-Functional Requirements

Non-functional requirements define the expected quality, efficiency, and practical usability of the proposed system.

| ID | Non-functional requirement | Priority | Acceptance criteria | Verification method |
|---|---|---|---|---|
| **NFR-01** | The system shall provide measurable classification performance for each NLP task. | Must | Performance is measured using appropriate metrics on a held-out test dataset. | Experimental evaluation |
| **NFR-02** | The system shall operate on the available computational resources. | Must | The implemented models can be loaded and executed in the target CPU-based environment. | Execution and resource testing |
| **NFR-03** | The system shall provide reproducible evaluation results. | Must | The dataset split, model configuration, evaluation metrics, and random seeds are documented where applicable. | Repeated experiments and documentation review |
| **NFR-04** | The system shall complete inference within a practical time for interactive use or batch evaluation. | Should | Average inference latency is measured and reported on the target hardware. | Performance testing |
| **NFR-05** | The system shall provide understandable and consistently structured outputs. | Must | Predictions follow a documented output schema, and each result can be associated with its input. | Output validation |
| **NFR-06** | The system should support maintainable and modular implementation. | Should | The three NLP components can be tested and updated independently without rewriting the entire pipeline. | Code inspection and integration testing |

### 6.3. Requirement Priorities

Requirements are classified into three priority levels:

- **Must:** Essential requirements necessary to demonstrate the main objectives of the diploma project.
- **Should:** Important requirements that improve usability or maintainability but do not prevent completion of the core research if omitted.
- **Could:** Optional enhancements that may be implemented if sufficient time and resources are available.

The mandatory requirements focus on accepting input, performing the three NLP tasks, integrating the components, handling invalid input, and evaluating the system. The user interface and additional usability improvements are considered secondary to the core research objectives.

### 6.4. Acceptance Criteria and Verification

Each mandatory requirement must have a corresponding verification method. Functional testing will establish whether the system performs the required operations, while integration testing will verify that the individual NLP components operate correctly within the cascaded architecture.

The classification components will be evaluated using a manually labeled test dataset that is separate from the training data. Accuracy and macro-averaged F1-score will be reported for language identification, token-level precision, recall, and F1-score will be used for code-switching detection, and macro-averaged F1-score will be used for sentiment analysis.

The system's computational feasibility will be assessed by measuring inference time and confirming that the models execute within the available hardware constraints. Invalid inputs will be tested to verify that the system handles them without unexpected failures.

Acceptance results will be documented in a test report containing the requirement identifier, test procedure, expected result, observed result, and pass/fail status. Any unmet mandatory requirement will be documented and addressed before the final system evaluation.

### 6.5. Requirement Traceability

A requirement traceability table will connect each requirement to its implementation and verification evidence. This will ensure that every mandatory requirement can be checked objectively.

| Requirement ID | Implementation component | Verification evidence |
|---|---|---|
| **FR-01** | Input processing module | Input validation test results |
| **FR-02** | Language-identification component | Language-identification metrics |
| **FR-03** | Code-switching detection component | Token-level evaluation results |
| **FR-04** | Sentiment-analysis component | Sentiment classification metrics |
| **FR-05** | Cascaded pipeline controller | Integration test results |
| **FR-06** | Output-processing module | Output schema validation |
| **FR-07** | Input validation and error handling | Negative test results |
| **NFR-01** | Model evaluation pipeline | Test-set metrics |

| | | |
|---|---|---|
| **NFR-02** | Model execution environment | Successful execution and resource measurements |
| **NFR-03** | Experimental configuration | Reproducibility documentation |
| **NFR-04** | Inference pipeline | Latency measurements |

The requirements will serve as the basis for implementation, testing, and final evaluation. Their acceptance status will be determined by recorded evidence rather than subjective judgment.

## 7. Architecture and Diagrams

### 7.1. Architectural Overview

The proposed system follows a modular architecture consisting of three main elements: an NLP processing pipeline, a REST API, and a native web interface. The system is designed to process Kazakh, Russian, and mixed Kazakh–Russian text through a sequence of connected NLP tasks.

The NLP pipeline is responsible for language identification, code-switching detection, and sentiment analysis. A pretrained XLM-based model will be investigated and adapted for the selected tasks, with the final implementation depending on the experimental results and the requirements of each task.

The REST API, implemented using FastAPI, will provide an interface between the NLP pipeline and external clients. It will receive text input, invoke the processing pipeline, and return structured analysis results. The native web interface will allow users to submit text and view the results without directly interacting with the API.

The architecture separates text processing, request handling, and user interaction to make the system easier to test, maintain, and extend. Persistent database storage is not a core requirement of the current design because the system primarily processes user-submitted text and returns analysis results. The need for persistent storage may be reconsidered if additional functionality requires it.

### 7.2. System Context Diagram

The system context diagram illustrates the proposed NLP system and its interactions with external users and systems. It defines the system boundary and identifies the main ways in which users and external applications can access its functionality.

The primary external actor is the user interacting with the native web interface. Developers of external applications, including educational platforms, may access the system through the REST API. Both access methods submit text for processing and receive the corresponding analysis results.

The system accepts Kazakh, Russian, and mixed Kazakh–Russian text and returns language identification, code-switching detection, and sentiment analysis results according to the capabilities of the implemented models.

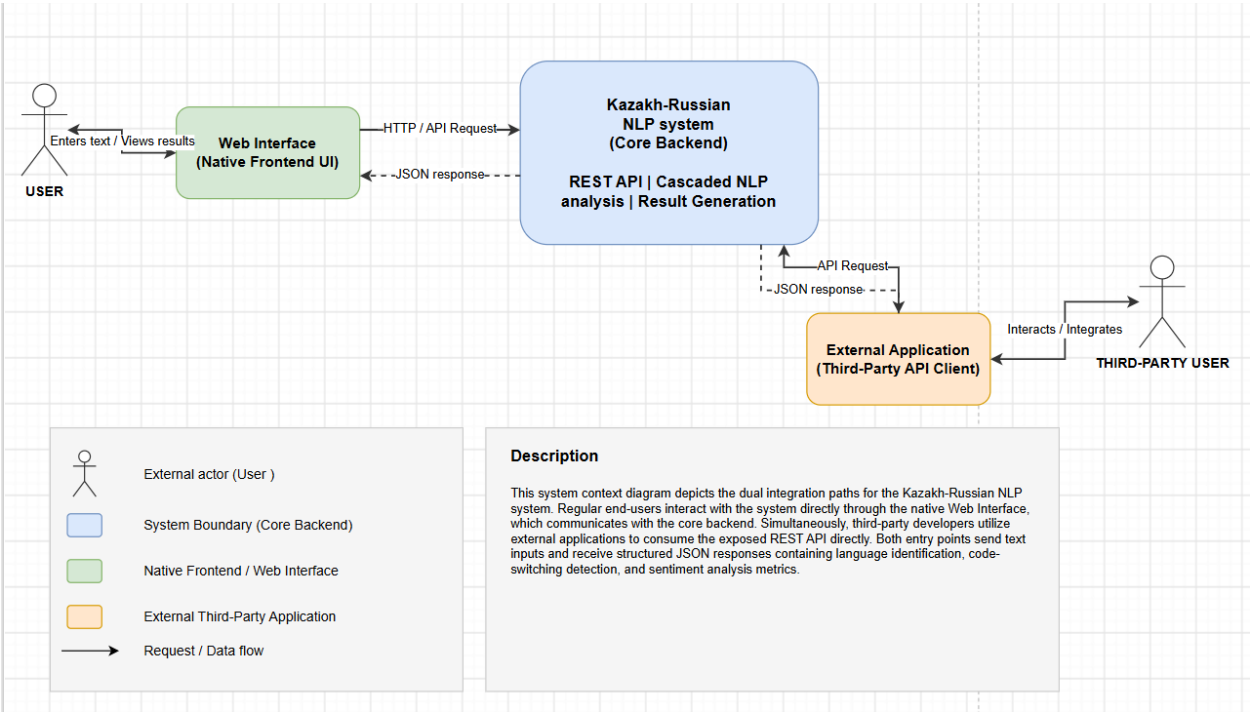



*Figure 1. System Context Diagram*

### 7.3. Use Case Diagram

The use case diagram describes the interactions between the system and its external actors. It identifies the main functions available to users and external applications.

The primary actor is the user, who can enter text through the native web interface, submit it for analysis, and view the resulting predictions. An external application can submit text through the REST API and receive structured analysis results for further processing.

The main use cases include:

- Submit text for analysis.
- Identify the language or languages present in the text.
- Detect the presence of code-switching.
- Analyze the sentiment of the submitted text.
- View analysis results through the web interface.
- Retrieve structured analysis results through the REST API.

The use cases represent the intended system functionality. Their final implementation will depend on the supported model outputs and the defined API and interface requirements.

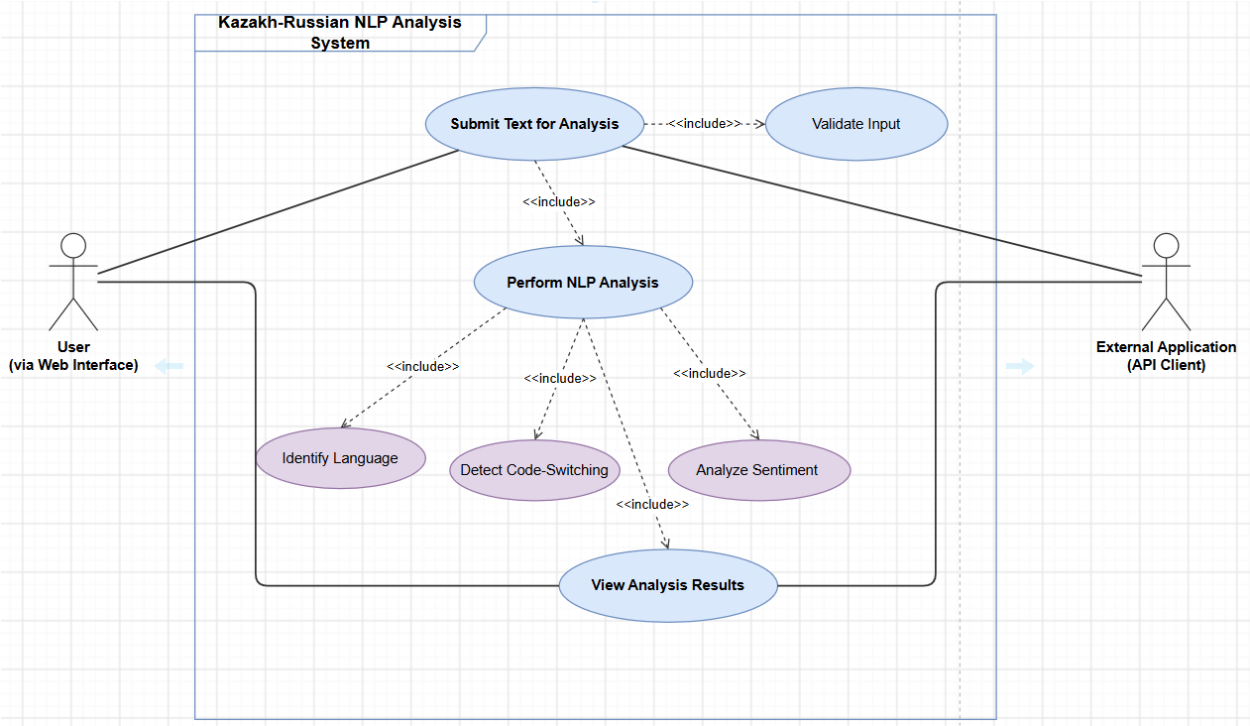



*Figure 2. Use Case Diagram*

### 7.4. Component Diagram

The component diagram describes the internal structure of the system and the relationships between its main software components.

The proposed architecture consists of the following components:

1. **Native Web Interface:** Provides a user-facing interface for entering text and displaying analysis results.
2. **REST API:** Handles incoming requests, validates input, invokes the NLP processing pipeline, and returns structured responses.
3. **NLP Processing Pipeline:** Coordinates the processing stages and combines their outputs.
4. **Language Identification Component:** Predicts whether the input contains Kazakh, Russian, or both languages, according to the supported classification scheme.
5. **Code-Switching Detection Component:** Determines whether the text contains language switching and produces the corresponding result.
6. **Sentiment Analysis Component:** Predicts the sentiment category supported by the trained model.
7. **Pretrained Model and Tokenizer:** Provide the underlying text representation and model inference functionality.

The web interface communicates with the backend through the REST API. The API passes validated input to the NLP pipeline, which coordinates the required processing

components and returns their results. The API then formats the output into a structured response for the requesting client.

The diagram will be refined during implementation to reflect the actual model configuration, processing dependencies, and component boundaries.

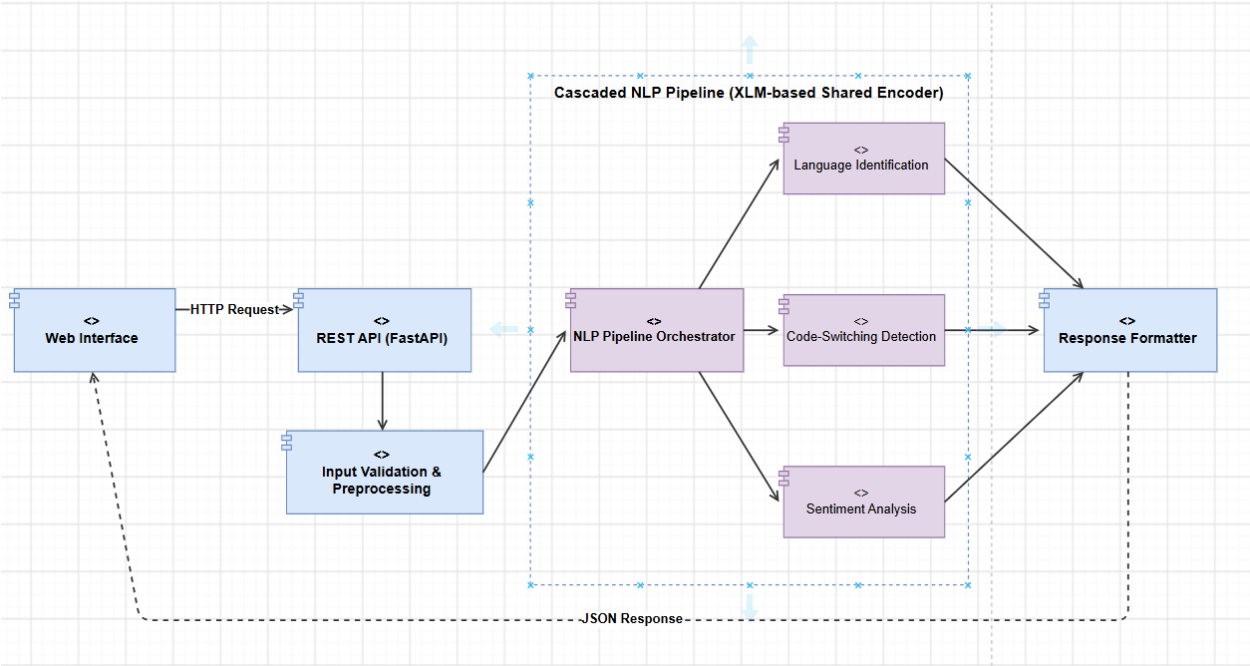



*Figure 3. Component Diagram (Draft)*

### 7.5. Alignment of Requirements, Components, Data, and Scenarios

The architecture is intended to maintain consistency between the project's functional requirements, software components, data inputs and outputs, and expected usage scenarios.

| Requirement | Responsible component | Input and output | Example scenario |
|---|---|---|---|
| Language identification | Language Identification Component | Input text → language prediction | A user submits a Russian or Kazakh sentence. |
| Code-switching detection | Code-Switching Detection Component | Input text → switching indicator and, if supported, language boundaries | A user submits a sentence containing Kazakh and Russian words. |
| Sentiment analysis | Sentiment Analysis Component | Input text → sentiment category | A user submits a comment expressing satisfaction or dissatisfaction. |

| | | | |
|---|---|---|---|
| Text submission through the web interface | Native Web Interface and REST API | User-entered text → API request | A user enters a message and requests analysis. |
| Programmatic integration | REST API | API request → structured JSON response | An educational platform submits student feedback for analysis. |
| Result presentation | Native Web Interface | API response → displayed predictions | A user views the results of text analysis. |

The main data flow begins when a user or external application submits text to the system. The REST API validates the request and passes the input to the NLP processing pipeline. The pipeline executes the configured analysis stages and collects their outputs. The API then returns the results as a structured response, which the web interface can display or an external application can process.

The architecture also supports testing at different levels. Individual NLP components can be evaluated using task-specific datasets and metrics, while integration tests can verify communication between the web interface, API, and processing pipeline. End-to-end tests can assess whether the complete system produces valid responses for Kazakh, Russian, and mixed-language inputs.

Overall, the three diagrams provide complementary views of the same system: the system context diagram defines its boundary and external interactions, the use case diagram describes its intended functionality, and the component diagram illustrates its internal organization. Together, they provide a consistent architectural foundation for implementation and evaluation.

## 8. UX / Prototype

The system will be developed as a REST API accompanied by a simple web interface. The prototype will demonstrate both API interaction, including request and response formats, and the user-facing interface for submitting text and viewing analysis results. The design will prioritize simplicity, consistency, and predictable behavior, ensuring that the system's NLP functionality can be demonstrated through both direct API requests and an intuitive web interface.

### 8.1 API and Web Interface Design

The system will provide a REST API and a simple web interface to make its NLP capabilities accessible through both programmatic integration and direct user interaction. The REST API will enable businesses, developers, and researchers to integrate the system into their own applications, including chatbots and text-processing platforms. A primary endpoint, POST /api/v1/analyze, will accept text in JSON format and return structured analysis results in JSON format. The API documentation will

describe the request parameters, response fields, and possible error messages. Interactive API documentation will be provided using FastAPI.

The web interface will provide a straightforward way for users to test the system without making API requests manually. Users will be able to enter Kazakh, Russian, or mixed-language text into an input field, submit it for analysis, and view the results in a clearly organized format. The results will present language identification, code-switching detection, and sentiment analysis, depending on the functionality implemented in the NLP pipeline. The interface will prioritize simplicity, readability, and ease of use, keeping the focus on demonstrating the system's NLP capabilities rather than providing complex website functionality.

Both the web interface and external applications will use the same underlying NLP processing pipeline through the REST API. This approach will maintain consistency in the analysis results and separate the user interface from the model implementation, making the system easier to test, maintain, and extend.

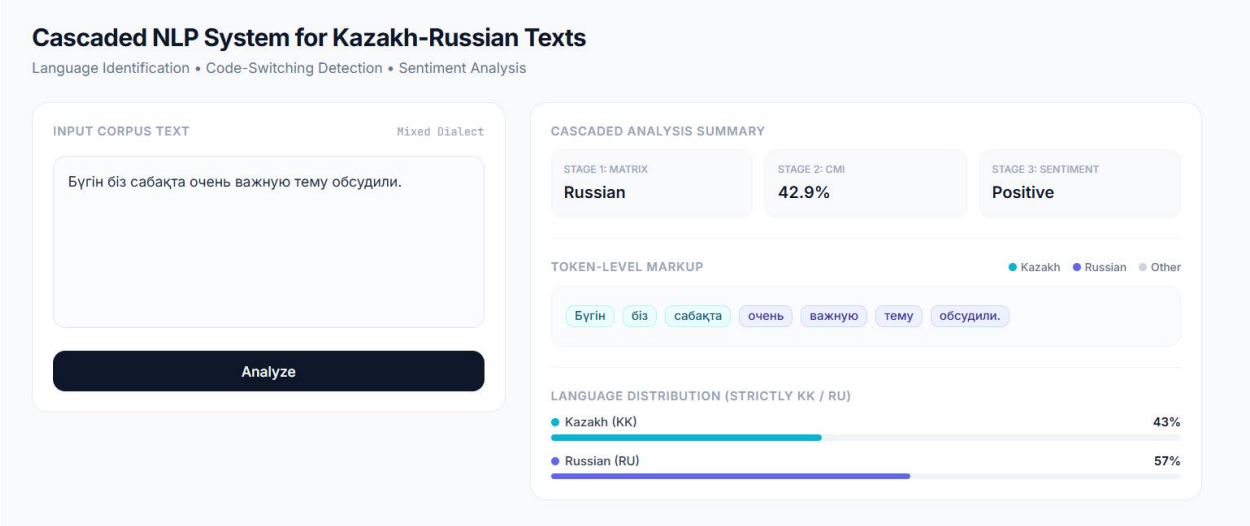



*Figure 4 – Mockup of the Kazakh–Russian NLP Web Interface*

### 8.2 Key User Scenarios

1. **Citizen or language learner:** The user enters a sentence containing Kazakh, Russian, or both languages into the web interface and receives the corresponding analysis results.
2. **Business or chatbot developer:** An external application sends a text request to the API and receives structured predictions that can be used for further processing.
3. **Researcher or developer:** The user accesses the API documentation to understand the available functionality, test requests, and evaluate the system's outputs.

### 8.3 Interaction Flow

The interaction process begins when a user or external application submits text. The system processes the input through the appropriate NLP stages, including language identification, code-switching detection, and sentiment analysis. The predictions are then returned to the user through the web interface or as a structured API response. The exact processing sequence will depend on the final cascade architecture.

### 8.4 Usability Verification

The prototype will be evaluated using representative user scenarios to determine whether users can submit text, understand the analysis results, and interpret the API responses correctly. Verification will include successful requests, mixed-language inputs, empty inputs, and invalid requests. Feedback will be used to identify unclear labels and improve the interface and documentation.

The proposed UX design will prioritize simplicity, clarity, and accessibility, while the API-first architecture will allow the NLP system to be integrated into a variety of external applications without requiring users to adopt a particular platform.

## 9. Verification and Evaluation Plan

### 9.1. Testing Strategy

The proposed cascaded NLP system will be evaluated through functional testing, integration testing, and experimental model evaluation. Functional testing will verify that each component performs its intended task, while integration testing will assess whether the language identification, code-switching detection, and sentiment analysis components operate correctly within the complete pipeline.

Experimental evaluation will measure the performance of the models using labeled Kazakh, Russian, and Kazakh-Russian code-switched text. The dataset will be divided into training, validation, and test sets. The test set will remain separate from model training and hyperparameter selection to provide an independent evaluation of the final system.

The experiments will investigate the suitability of pretrained XLM-RoBERTa models for the three NLP tasks and assess whether the proposed cascaded architecture provides practical benefits compared with simpler baseline approaches.

### 9.2. Evaluation Metrics

Different metrics will be used because the three components perform different classification tasks.

| Component | Evaluation metrics | Purpose |
|---|---|---|
| **Language identification** | Accuracy, macro-F1 | Measures the correctness of language predictions |

| | | |
|---|---|---|
| **Code-switching detection** | Token-level precision, recall, macro-F1 | Measures the accuracy of language labels assigned to individual tokens |
| **Sentiment analysis** | Accuracy, macro-F1, confusion matrix | Measures sentiment classification performance and class-specific errors |
| **Complete pipeline** | End-to-end evaluation, inference latency | Measures overall system behavior and processing efficiency |

Macro-F1 will be emphasized because the dataset may contain unequal numbers of Kazakh, Russian, and code-switched examples, as well as an imbalance between sentiment classes.

Functional and integration tests will additionally verify valid input processing, structured output generation, and appropriate handling of empty or invalid input.

### 9.3. Baseline Solutions and Comparative Evaluation

The proposed system will be compared with simpler approaches to determine whether the selected models and architecture provide measurable benefits.

TF-IDF with Logistic Regression will serve as a lightweight baseline for sentence-level classification tasks where its formulation is appropriate. For token-level code-switching detection, a suitable baseline will be developed using token-level features and a lightweight classifier. Pretrained XLM-RoBERTa models fine-tuned for individual tasks will also be evaluated to establish the performance of the model components before integrating them into the cascade.

The evaluation will distinguish between two comparisons. First, individual models will be compared to assess the contribution of the selected model architecture. Second, the cascaded pipeline will be compared with independent task models to investigate whether integrating the components improves overall performance.

All approaches will be evaluated using the same held-out test data and consistent metrics wherever the tasks are directly comparable. Results will be reported without assuming in advance that the proposed system will outperform every baseline.

### 9.4. Measurement Conditions and Experimental Procedure

The models will be evaluated using a labeled dataset containing Kazakh, Russian, and Kazakh-Russian code-switched text. The data will be divided into training, validation, and test sets before model development. Where possible, duplicate or near-duplicate sentences will be prevented from appearing across different splits to reduce data leakage.

The training set will be used to fine-tune the models, while the validation set will support model selection and hyperparameter adjustment. The held-out test set will be used for final performance measurement.

The experiments will be conducted in the available CPU-based development environment, with the model configuration, dataset version, data splits, and evaluation procedures documented. Inference latency will be measured under specified hardware

and input conditions. The number of evaluated samples and relevant computational constraints will be reported alongside the results.

If the available dataset is too small to support reliable conclusions, its limitations will be documented and additional labeled data will be collected where feasible.

### 9.5. Demonstration of Project Aim Achievement

The project aim will be assessed by verifying that the implemented system can process Kazakh, Russian, and code-switched text through the three intended NLP tasks.

Achievement will be demonstrated by:

1. Implementing language identification, token-level code-switching detection, and sentiment analysis components based on pretrained XLM-RoBERTa models.
2. Integrating the components into a functioning cascaded pipeline.
3. Evaluating each task on a held-out, manually labeled test dataset using the specified metrics.
4. Comparing the results with appropriate baseline approaches under consistent experimental conditions.
5. Reporting the system's performance, inference time, limitations, and observed errors.

The project will be considered successfully demonstrated when the three components operate within the implemented pipeline, mandatory functional tests pass, and the experimental evaluation provides documented results for all three tasks. The research conclusions will be based on the measured results rather than predetermined performance claims.

The final report will include the evaluation tables, baseline comparisons, and an analysis of whether the cascaded architecture is appropriate for Kazakh-Russian text processing.

## 10. Project Management and risks

### 10.1 Project Management Approach

The project is being developed individually; therefore, project management focuses on personal task organization, progress tracking, version control, and risk management. Development tasks, milestones, and project progress are organized and tracked through a dedicated **YouTrack** project board https://kazakh-russian-nlp-system.youtrack.cloud/projects/KRN. Each task is assigned a specific priority, description, and status. Additionally, Git is used to maintain version history and track source code changes, with individual commits and feature branches explicitly cross-referenced to their corresponding YouTrack task IDs (e.g., KRN-23) to ensure full development traceability throughout the project lifecycle.

### 10.2 Git Repository and Version Control

A Git repository will be used to manage the project's source code, dataset preparation scripts, model training and evaluation procedures, API implementation, and technical documentation. Version control will support the tracking of development progress and maintain a record of significant changes throughout the project.

The project repository is hosted on GitHub and is available at: **Kazakh-Russian NLP System - GitHub Repository.**

### 10.3 Project Milestones

The project will be divided into the following milestones:

1. **Dataset Preparation:** Collect, organize, clean, and preprocess the data required for language identification, code-switching detection, and sentiment analysis.
2. **Model Development:** Fine-tune and configure the selected pretrained XLM model for the project's NLP tasks.
3. **Model Evaluation:** Evaluate model performance using appropriate metrics, including accuracy, precision, recall, and F1-score.
4. **API Implementation:** Develop a REST API using FastAPI to provide access to the trained NLP model.
5. **Testing and Validation:** Test the model and API using Kazakh, Russian, and mixed Kazakh–Russian text examples.
6. **Documentation and Finalization:** Document the system architecture, implementation process, evaluation results, and limitations.

### 10.4 Allocation of Responsibilities

Since the project is completed by a single student, all project responsibilities are assigned to the developer. These responsibilities include dataset preparation, model development, training, evaluation, API implementation, testing, documentation, and project planning. A separate team responsibility allocation is therefore not applicable.

### 10.5 Risk Management

Several potential risks have been identified:

- **Limited dataset size:** An insufficient number of training examples may affect the model's ability to generalize. This risk will be addressed by cleaning and expanding the dataset where possible and evaluating the model on held-out test data.
- **Limited computational resources:** Training and fine-tuning transformer models on CPU-only hardware may require considerable time. The project will use a pretrained XLM model and manage training experiments to reduce unnecessary computational overhead.

- **Model performance limitations:** The model may perform less accurately on code-switched text or sentences containing ambiguous language and tone. Performance will be evaluated separately where possible, and limitations will be documented.
- **Integration difficulties:** Connecting the trained model to the API may introduce compatibility or preprocessing issues. These will be addressed through incremental implementation and API testing.

### 10.6 Technical Dependencies

The project depends on Python and relevant machine learning and development libraries, including PyTorch, Hugging Face Transformers, pandas, scikit-learn, and FastAPI. It also depends on the availability of a suitable pretrained XLM model, prepared datasets, and sufficient computational resources for training and evaluation. Dependency versions will be managed to maintain compatibility and reproducibility.

### 10.7 Task Tracking and Progress Monitoring

Tasks and milestones will be tracked using a personal task list and the Git repository. Progress will be monitored against the planned milestones, and the task list will be updated as development proceeds.

## 11.Supervisor's feedback

The proposal is correct and ready for submission. No further revisions are required.

## References

**1.** Conneau, A., Khandelwal, K., Goyal, N., et al. (2020). Unsupervised Cross-lingual Representation Learning at Scale. Proceedings of ACL 2020. https://doi.org/10.18653/v1/2020.acl-main.747

**2.** Khanuja, S., Dandapat, S., Srinivasan, A., Sitaram, S., & Choudhury, M. (2020). GLUECoS: An Evaluation Benchmark for Code-Switched NLP. Proceedings of ACL 2020. https://doi.org/10.18653/v1/2020.acl-main.329

**3.** Ranjan, S., Mekala, D., & Shang, J. (2022). Progressive Sentiment Analysis for Code-Switched Text Data. Findings of EMNLP 2022. https://doi.org/10.18653/v1/2022.findings-emnlp.82

**4.** Agarwal, M., Alam, M. M. I., & Anastasopoulos, A. (2023). LIMIT: Language Identification, Misidentification, and Translation using Hierarchical Models in 350+ Languages. Proceedings of EMNLP 2023. https://doi.org/10.18653/v1/2023.emnlp-main.895

**5.** Burchell, L., Birch, A., Thompson, R., & Heafield, K. (2024). Code-Switched Language Identification is Harder Than You Think. Proceedings of EACL 2024. https://doi.org/10.18653/v1/2024.eacl-long.38

**6.** Burchell, L., Birch, A., Bogoychev, N., & Heafield, K. (2023). An Open Dataset and Model for Language Identification. Proceedings of ACL 2023. https://doi.org/10.18653/v1/2023.acl-short.75

**7.** Yeshpanov, R., & Varol, H. A. (2024). KazSAnDRA: Kazakh Sentiment Analysis Dataset of Reviews and Attitudes. Proceedings of LREC-COLING 2024. https://doi.org/10.48550/arXiv.2403.19335

**8.** Sharma, G., Chinmay, R., & Sharma, R. (2023). Late Fusion of Transformers for Sentiment Analysis of Code-Switched Data. Findings of EMNLP 2023. https://doi.org/10.18653/v1/2023.findings-emnlp.430

**9.** Etori, N. A., & Gini, M. L. (2024). RideKE: Leveraging Low-resource, User-generated Twitter Content for Sentiment and Emotion Detection in Kenyan Code-switched Dataset. Proceedings of WASSA 2024. https://doi.org/10.18653/v1/2024.wassa-1.19

**10.** Gupta, A., Menghani, S., Rallabandi, S. K., & Black, A. W. (2021). Unsupervised Self-Training for Sentiment Analysis of Code-Switched Data. Proceedings of the Fifth Workshop on Computational Approaches to Linguistic Code-Switching. https://doi.org/10.18653/v1/2021.calcs-1.13

**11.** Samanta, B., Ganguly, N., & Chakrabarti, S. (2019). Improved Sentiment Detection via Label Transfer from Monolingual to Synthetic Code-Switched Text. Proceedings of ACL 2019. https://doi.org/10.18653/v1/P19-1343

**12.** Zhang, R., Cahyawijaya, S., Cruz, J. C. B., Winata, G., & Aji, A. F.(2023).Multilingual Large Language Models Are Not (Yet) Code-Switchers.Proceedings of EMNLP 2023. https://doi.org/10.18653/v1/2023.emnlp-main.774

## Appendix A. Interview Transcripts

### A.1. Interview with Participant P1

**Participant profile:** Primarily Russian-speaking individual with limited Kazakh proficiency who encounters Kazakh in everyday online communication.

**Question 1.** How does your level of proficiency in Kazakh and Russian affect your ability to understand or write texts that contain both languages?

**Response:** I am more comfortable communicating in Russian because my Kazakh vocabulary is limited. When I encounter mixed-language messages, I can usually understand the Russian parts but sometimes struggle with Kazakh words and expressions.

**Question 2**. Have you encountered situations where identifying the language or understanding the sentiment and tone of a mixed Kazakh–Russian text was difficult? Please describe an example.

**Response:** Yes. Sometimes I understand the general meaning of a message but cannot tell whether the person is being serious, sarcastic, or annoyed. This becomes more difficult when the message contains unfamiliar Kazakh expressions.

**Question 3.** Would an NLP system that identifies languages, detects code-switching, and analyzes sentiment and tone be useful to you? How would you use it?

**Response:** Such a system could help me identify which parts of a message are in Kazakh and Russian. Sentiment and tone analysis could also provide additional context when I am unsure how to interpret someone's message.

### A.2. Interview with Participant P2

**Participant profile:** Individual who regularly uses Russian and has limited Kazakh proficiency.

**Question 1.** How does your level of proficiency in Kazakh and Russian affect your ability to understand or write texts that contain both languages?

**Response:** I primarily use Russian in written communication. I understand common Kazakh words, but longer sentences or texts that frequently switch between the two languages can be difficult to follow.

**Question 2.** Have you encountered situations where identifying the language or understanding the sentiment and tone of a mixed Kazakh–Russian text was difficult? Please describe an example.

**Response:** I have encountered mixed-language posts on social media where I could understand individual words but not the entire message. I also find it difficult to recognize the emotional meaning of some expressions without understanding their context.

**Question 3.** Would an NLP system that identifies languages, detects code-switching, and analyzes sentiment and tone be useful to you? How would you use it?

**Response:** I would find the system useful for analyzing social media posts and comments. Identifying the languages and estimating whether a message is positive, negative, or neutral could help me understand the content more quickly.

### A.3. Interview with Participant P3

**Participant profile:** Individual who is more comfortable communicating in Kazakh and encounters Russian in online and educational communication.

**Question 1.** How does your level of proficiency in Kazakh and Russian affect your ability to understand or write texts that contain both languages?

**Response:** I am more confident in Kazakh than in Russian. I can understand common Russian expressions, but I sometimes have difficulty with longer Russian sentences, specialized vocabulary, and messages that frequently switch between the two languages.

**Question 2.** Have you encountered situations where identifying the language or understanding the sentiment and tone of a mixed Kazakh–Russian text was difficult? Please describe an example.

**Response:** Yes. When a message contains unfamiliar Russian words mixed with Kazakh sentences, I may understand the general topic but miss some details. It can also be difficult to interpret the writer's intended tone.

**Question 3.** Would an NLP system that identifies languages, detects code-switching, and analyzes sentiment and tone be useful to you? How would you use it?

**Response:** The system could help me understand mixed-language communication by identifying which language is used and providing sentiment and tone predictions. It could be particularly useful when reading online discussions or educational materials.

### A.4. Interview with Participant P4

**Participant profile:** Educational platform developer or representative interested in integrating the proposed NLP model into an educational platform operating in Kazakhstan.

**Question 1.** How does your level of proficiency in Kazakh and Russian affect your ability to understand or write texts that contain both languages?

**Response:** Our platform operates in a multilingual environment where Kazakh and Russian are both relevant. Students may use either language or combine them when writing comments, submitting feedback, and communicating through educational tools.

**Question 2.** Have you encountered situations where identifying the language or understanding the sentiment and tone of a mixed Kazakh–Russian text was difficult? Please describe an example.

**Response:** Mixed-language content can make automated text processing less reliable, particularly when a sentence contains words from both languages. Understanding sentiment could also help us organize student feedback and identify recurring concerns.

**Question 3.** Would an NLP system that identifies languages, detects code-switching, and analyzes sentiment and tone be useful to you? How would you use it?

**Response:** I would be interested in integrating this model into our educational platform through a REST API. For example, when students submit feedback or course evaluations, the platform could send the text to the model and receive language identification, a code-switching indicator, and sentiment and tone predictions. This could help us analyze feedback in both languages without requiring the platform to implement its own NLP models.

## Appendix B. Interview Recording

**Google Drive folder containing all interview recordings:** Open Interview Recordings